TECHNICAL ISSUES

- Previously, unreferenced objects were retained indefinitely, causing a continuous upward trajectory in heap consumption that ultimately led to system crashes. Following the optimization, memory utilization now exhibits a stable, cyclical pattern. While heap usage increases during active gameplay, it is immediately reclaimed and returned to the baseline once a garbage collection cycle executes.
- Verification and Testing - This behavioral correction was validated using VisualVM. By manually invoking the GC during profiling sessions, we observed the heap allocation immediately drop back to the initial baseline. This confirms that the application is correctly releasing object references. The upward slope observed in the telemetry graphs no longer represents a memory leak, rather, it reflects standard, deferred JVM cleanup behavior. Profiling confirms that no illegitimate objects are being retained in the heap, and all transient data is now fully eligible for collection.
- Configuration and Risk Mitigation - To guarantee a predictable memory footprint for the end-user, we have configured the executable JAR with a maximum heap restriction of 3GB (-Xmx3GB).
- This constraint is a proactive resource management strategy, not a performance limitation. Setting this threshold forces the JVM to optimize its cleanup intervals, rather than allowing deferred data to accumulate indefinitely, the JVM will automatically initiate a GC cycle as allocation approaches the 3GB ceiling. This boundary ensures the application remains highly stable and performant within standard hardware limits, without compromising visual quality, asset rendering, or frame rates.

Comprehensive Explanation of the Game Architecture, Movement System, Level Logic, and Class Connections

1. Introduction

This project is a lane-based Java game architecture that uses object-oriented programming (OOP), game loops, collision systems, entity management, rendering systems, and level generation.

The codebase is organized into multiple classes, where each class has a specific responsibility.

Some Classes:

- `GameObject` → Base class for all entities

- `Player` → Controls the character
- `Obstacle` → Moving hazards
- `Platform` → Moving rideable objects
- `Coin` → Collectibles
- `LevelManager` → Creates and manages level contents
- `LevelSetup` → Starts and initializes levels
- `CollisionSystem` → Detects collisions
- `GamePanel` → Main game rendering and update area
- `GameStateManager` → Controls PLAYING, PAUSED, GAME OVER states

This architecture follows:

- Encapsulation
- Inheritance
- Polymorphism
- Separation of concerns
- Modular game design

---

2. The Core Foundation: GameObject Class

```
public abstract class GameObject
```

This is the most important parent class in the entire game. Every visible object in the game inherits from this class. All of them automatically receive:

- Position
- Size
- Speed
- Collision bounds
- Active/inactive state

---

3. Variables Inside GameObject

Position Variables

```
protected int x, y;
```

These determine where the object exists on the screen.

- `x` = horizontal position
- `y` = vertical position

`x = 200` means object is 200 pixels from the LEFT side.

`y = 100` means object is 100 pixels from the TOP.

Size Variables

```
protected int width, height;
```

These determine how large the object is.

```
width = 50
height = 50
```

The object occupies a 50x50 pixel rectangle.

Speed Variable

```
protected float speed;
```

Controls how fast the object moves.

Active State

```
private boolean isActive;
```

Determines whether the object should update, draw, and collide

If `isActive = false`, the object is basically disabled.

Used for:

- collected coins
- destroyed entities
- removed obstacles

---

4. Constructor of GameObject

```
public GameObject(int x, int y, int width, int height, float speed)
```

“Save the starting position, size, and speed.”

---

5. Collision Detection

```
public Rectangle getBounds()
```

Returns:

```
new Rectangle(x, y, width, height)
```

This creates a collision box. The collision system checks:

```
player.getBounds().intersects(obstacle.getBounds())
```

If rectangles overlap, a collision happens.

---

6. Abstract Methods

```
public abstract void move();
public abstract void update();
public abstract void draw(Graphics g);
public abstract void onCollide(GameObject other);
```

---

7. How Player Movement Works (STEP-BY-STEP)

STEP 1 — Keyboard Input

A key listener detects keyboard presses.

Example: `if(key == KeyEvent.VK_UP)`. This means: “The UP arrow was pressed.”

STEP 2 — Player Movement Function Runs

Inside the player class: `y -= laneHeight;`

In Java graphics:

- TOP = smaller y
- BOTTOM = larger y

So: `y -= something` means move UP.

STEP 3 — Game Loop Updates

Every frame: `update()` runs.

The player now uses the new coordinates.

STEP 4 — Rendering

The rendering system calls `draw(Graphics g)`, which does:

```
g.drawImage(playerImage, x, y, width, height, null);
```

Now the player appears in the new location.

---

8. LEFT and RIGHT Movement

RIGHT

```
x += columnWidth;
```

Example:

`x = 200`, `columnWidth = 80`

After moving: `x = 280` Player moves RIGHT.

LEFT

```
x -= columnWidth;
```

Example:

```
x = 280
```

After movement: `x = 200`. Player moves LEFT.

---

9. Why the Player Moves exactly Into Grid Spaces

The game uses:

- lanes
- columns

This creates a grid. This guarantees:

- perfect alignment

- clean movement
- consistent positioning

---

10. LevelManager — The Brain of the Level

```
public class LevelManager
```

This class controls:

- lanes
- obstacle spawning
- platform spawning
- coin spawning
- movement updates
- respawning
- background
- scaling
- level difficulty

---

11. Important Variables in LevelManager

Lane System

```
private static final int LANE_COUNT = 10;
```

The game has 10 horizontal lanes.

Column System

```
private static final int COLUMN_COUNT = 10;
```

The game has 10 vertical columns.

Together:

```
10 x 10 grid
```

---

12. computeLanes() — HOW THE GRID IS CREATED

```
private void computeLanes()
```

It mathematically divides the screen.

STEP-BY-STEP PROCESS

STEP 1 — Divide Screen Height

```
int lane0Height = screenHeight / 6;
```

The top lane is special.

STEP 2 — Remaining Height

```
int remainingHeight = screenHeight - lane0Height;
```

Everything else is divided equally.

STEP 3 — Calculate Lane Height

```
laneHeight = remainingHeight / (LANE_COUNT - 1);
```

Now every lane has equal size.

STEP 4 — Store Y Positions

```
laneY[i] = lane0Height + (i - 1) * laneHeight;
```

This stores the TOP coordinate of each lane.

Example: `laneY[5] = 300` means lane 5 starts at y=300.

STEP 5 — Calculate Columns

```
columnWidth = screenWidth / COLUMN_COUNT;
```

Each column gets equal width.

STEP 6 — Store Column X Positions

```
columnX[i] = i * columnWidth;
```

This stores where every column begins.

---

13. Lane Classification System

```
private void initLaneClassification()
```

This determines what each lane does.

Platform Lanes

```
isPlatformLane[i] = i >= 1 && i <= 3;
```

Lanes 1–3 contain moving platforms.

Safe Lane

```
isSafeLane[i] = i == 4;
```

Lane 4 is safe. No obstacles.

Obstacle Lanes

```
isObstacleLane[i] = i >= 5 && i <= 8;
```

Lanes 5–8 contain hazards.

---

14. How Obstacles Spawn

STEP 1 — spawnObstacles() Runs

```
private void spawnObstacles()
```

This creates ALL moving hazards.

STEP 2 — Loop Through Every Lane

```
for(int lane = 0; lane < LANE_COUNT; lane++)
```

Every lane is checked.

STEP 3 — Ignore Non-Obstacle Lanes

```
if(!isObstacleLane[lane])
    continue;
```

Only obstacle lanes are processed.

STEP 4 — Determine Direction

```
Direction dir = (lane % 2 == 0)
    ? Direction.RIGHT
    : Direction.LEFT;
```

This alternates directions.

Meaning:

- even lanes → RIGHT
- odd lanes → LEFT

This creates traffic-like movement.

STEP 5 — Load Obstacle Image

```
Image img = AssetManager.getInstance().getObstacleImage(type);
```

AssetManager retrieves sprite images.

This connects:

- LevelManager
- AssetManager

STEP 6 — Calculate Size

```
Dimension d = scaleToFitLane(img, columnWidth, laneHeight);
```

This scales images to fit lanes properly.

STEP 7 — Create Groups

```
List<List<Obstacle>> laneGroups
```

Obstacles are grouped, so they respawn together.

STEP 8 — Create Each Obstacle

```
Obstacle o = new Obstacle(...)
```

Constructor receives:

- x
- y
- width
- height
- lane
- speed
- direction
- type

STEP 9 — Store Obstacle

```
obstacles.add(o);
```

Adds obstacle into the master obstacle list.

---

15. How Obstacles Move

Every frame:

```
update()
```

runs.

Inside:

```
o.update();
```

calls obstacle update logic.

Inside Obstacle class:

```
move();
```

then:

```
if(direction == Direction.RIGHT)
    x += speed;
else
    x -= speed;
```

---

17. How Coins Work

Coins are generated by:

```
spawnCoins()
```

Coin Placement Logic

Platform Lane

Coin attaches to a platform.

```
coin.attachToPlatform(pf);
```

Meaning: When platform moves, coin moves too.

Obstacle Lane

Coin is placed carefully. This prevents overlap:

```
obs.getBounds().intersects(coin.getBounds())
```

---

18. How Respawning Works

STEP 1 — Object Leaves Screen

`o.isOffScreen(screenWidth)` checks if obstacle moved beyond screen boundary.

STEP 2 — Entire Group Checked

`isGroupOffScreen(group)` All obstacles in group must leave screen.

STEP 3 — Respawn Triggered

`respawnObstacleGroup(group)` runs.

STEP 4 — Reposition Offscreen

Example: `baseX = -d.width - jitter;` Obstacle starts OFFSCREEN.

Then moves back into view. This creates infinite looping traffic.

---

19. Game Update Loop

`public void update()` Runs repeatedly. Usually `60 times per second`

Update Order

1. Update Obstacles

```
o.update();
```

2. Respawn Obstacles

```
respawnObstacleGroup(group);
```

3. Update Platforms

```
p.update();
```

4. Respawn Platforms

5. Update Coins

```
c.update();
```

Everything moves every frame.

---

20. Rendering System

```
public void draw(Graphics g)
```

Responsible for VISUAL OUTPUT

---

21. LevelSetup — The Level Initializer

```
public class LevelSetup
```

Responsible for:

- starting levels
- preparing UI
- spawning player
- creating level manager
- attaching resize listeners
- starting threads

---

22. startLevel()

`public void startLevel(...)` This method starts everything.

FULL STEP-BY-STEP START PROCESS

STEP 1 — Initialize State

```
stateManager.setPlayerAlive(true);
```

Player is alive.

STEP 2 — Set Game State

```
stateManager.setState(GameState.PLAYING);
```

Game officially enters PLAYING mode.

STEP 3 — Set Player

```
gamePanel.setPlayer(selectedPlayer);
```

GamePanel now knows who to render.

STEP 4 — Reset Lives

```
selectedPlayer.resetLives();
```

Player gets fresh lives.

STEP 5 — Load Music

```
sound.playBGM("game");
```

Starts background music.

STEP 6 — Initialize LevelManager

```
initLevelManager(map);
```

Creates:

- lanes
- obstacles
- platforms
- coins
- background

STEP 7 — Spawn Player

```
spawnPlayer(...)
```

Places player bottom center.

STEP 8 — Build UI

```
buildUI(selectedPlayer);
```

Creates:

- HUD
- Pause menu
- Resize handling

STEP 9 — Start Threads

```
gamePanel.startThreads();
```

Without this:

- no movement
- no updates
- no rendering
- frozen game

---

23. Player Spawn Logic

`spawnPlayer(Player player, LevelManager lm)`

STEP 1 — Choose Spawn Column

`spawnCol = lm.getColumnCount() / 2;` Player spawns middle column.

STEP 2 — Choose Spawn Lane

`spawnLane = lm.getLaneCount() - 1;` Bottom lane.

STEP 3 — Calculate Position

```
spawnX = ...
spawnY = ...
```

Centers player in grid cell.

STEP 4 — Apply Position

`player.setPosition(spawnX, spawnY);` Player now officially exists in the world.

---

24. Collision and Losing Lives

When player touches obstacle:

`loseLife();` runs.

```java
public void loseLife() {
    if (invincibilityFrames > 0)
        return;

    lives--;
    invincibilityFrames = 60;
}
```

STEP-BY-STEP

STEP 1 — Check Invincibility

```
if(invincibilityFrames > 0)
```

If player recently got hit, ignore damage.

STEP 2 — Reduce Life

`lives--;` Subtracts 1.

---

25. How Game Over Happens

```
if(player.getLives() <= 0)
```

then: `stateManager.setState(GameState.GAME_OVER);`

Game transitions to:

- game over screen
- stop gameplay
- stop movement

---

27. Full Runtime Flow of the Game

GAME STARTS

```
GameLauncher
    ↓
LevelSetup.startLevel()
```

LEVEL INITIALIZED

```
LevelManager.loadLevel()
```

OBJECTS SPAWN

```
spawnObstacles()
spawnPlatforms()
spawnCoins()
spawnPlayer()
```

GAME LOOP STARTS

```
gamePanel.startThreads()
```

EVERY FRAME

```
update()
    ↓
move objects
    ↓
collision detection
    ↓
render graphics
```

PLAYER INPUT

```
keyboard input
    ↓
player x/y changes
    ↓
new position rendered
```

COLLISION

```
player hits obstacle
    ↓
loseLife()
    ↓
lives decrease
```

GAME OVER

```
lives = 0
    ↓
GameState.GAME_OVER
```

---

28. Architecture

Separation of Concerns

Each class has one responsibility.

Reusability

GameObject logic reused by all entities.

Scalability

Easy to add:

- new enemies
- new maps
- new collectibles

Maintainability

Bugs easier to fix.

Encapsulation

Data protected through getters/setters.

Polymorphism

Different objects implement their own movement/update logic.

---

29. Methods Summary

Method	Purpose
`computeLanes()`	Creates grid system
`spawnObstacles()`	Creates moving hazards
`spawnPlatforms()`	Creates moving rideable objects
`spawnCoins()`	Places collectibles
`update()`	Main game logic loop
`draw()`	Renders graphics
`respawnObstacleGroup()`	Recycles obstacles
`startLevel()`	Starts entire game level
`spawnPlayer()`	Places player in map
`loseLife()`	Handles damage
`getBounds()`	Collision detection

---

30. Final Technical Analysis

This game architecture demonstrates:

- Advanced object-oriented programming
- Real-time game loop design
- Dynamic level management
- Collision systems
- Rendering systems
- State management
- Entity management
- Grid-based movement systems
- Procedural spawning systems
- Resize-responsive UI/gameplay systems

The system is modular, scalable, and maintainable.

The design follows many professional game programming principles used in:

- Java 2D games
- Indie game engines
- Entity systems
- Arcade lane-based games
- Real-time update/render architectures